STUDENT MANAGEMENT SYSTEM IN C++

A PROJECT REPORT

Submitted by

KIRAN RAKSHIT

UID: 25BCD10098

in partial fulfillment for the award of the degree of

BACHELOR IN BCA DATA SCIENCE HONOURS WITH RESEARCH

IN

BCA DATA SCIENCE



Chandigarh University

October 2026

BONAFIDE CERTIFICATE

Certified that this project report titled "STUDENT MANAGEMENT SYSTEM IN C++" is the bonafide work of "KIRAN RAKSHIT (UID: 25BCD10098)" submitted for the subject Object Oriented Programming Language. The project work has been prepared for academic learning under the guidance of the concerned faculty member.

_____	_____
SIGNATURE OF HOD	**SIGNATURE OF SUPERVISOR**
Name: _____ Department: BCA Data Science	Name: _____ Designation: _____

Submitted for the project viva-voce examination held on: _____

_____	_____
INTERNAL EXAMINER	**EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Chandigarh University and the faculty members of the BCA Data Science department for providing me with the opportunity to prepare this project report on "Student Management System in C++" for the subject Object Oriented Programming Language.

I am thankful to my subject teacher for the guidance and support provided during the planning and development of this project. The concepts of classes, objects, encapsulation, functions, arrays, and file handling helped me understand how object-oriented programming can be applied to a practical problem.

I would also like to thank my family and friends for their encouragement. Finally, I acknowledge the learning resources and documentation that helped me understand C++ syntax and program design.

Kiran Rakshit

UID: 25BCD10098

ABSTRACT

The Student Management System in C++ is a console-based application designed to maintain basic student records in an organized and efficient way. Manual record keeping can lead to duplication, misplaced information, and delays when searching or updating student details. This project demonstrates how a simple computer program can reduce these difficulties by providing a structured method to add, display, search, update, and delete student records.

The system is developed using C++ and object-oriented programming concepts. A Student class represents each student and stores information such as roll number, name, course, and marks. Member functions perform the required operations, while a menu-driven interface allows the user to select an action. File handling is used to store records so that the information can remain available after the program is closed.

The project is intended for learning and demonstration. It is suitable for a small academic setting and can be extended in the future with login authentication, a database, attendance management, grade calculation, and a graphical user interface. The report explains the problem, background, design, implementation, validation, conclusion, references, and user instructions.

Keywords

C++, Object-Oriented Programming, Student Records, Classes, Objects, Encapsulation, File Handling, CRUD Operations.

GRAPHICAL ABSTRACT

User → Main Menu → Add / Display / Search / Update / Delete → File Storage → Output

The application accepts a user's choice through a menu, performs the selected operation on student records, and saves the records in a local data file. The same file is read when records need to be displayed or searched.

ABBREVIATIONS

Abbreviation	Meaning
OOP	Object-Oriented Programming
CRUD	Create, Read, Update, Delete
UID	University Identification Number
IDE	Integrated Development Environment
I/O	Input/Output
STL	Standard Template Library

SYMBOLS

This project does not use mathematical or engineering symbols that require a separate symbol list. Common programming symbols used in the source code include: ; (statement terminator), {} (block delimiters), () (function or expression delimiters), == (equality comparison), = (assignment), and & (reference operator).

TABLE OF CONTENTS

ABSTRACT
GRAPHICAL ABSTRACT
ABBREVIATIONS
SYMBOLS
CHAPTER 1. INTRODUCTION
1.1 Identification of Client / Need / Relevant Contemporary Issue
1.2 Identification of Problem
1.3 Identification of Tasks
1.4 Timeline
1.5 Organization of the Report
CHAPTER 2. LITERATURE REVIEW / BACKGROUND STUDY
2.1 Timeline of the Reported Problem
2.2 Existing Solutions
2.3 Feature-Based Review
2.4 Review Summary
2.5 Problem Definition
2.6 Goals / Objectives
CHAPTER 3. DESIGN FLOW / PROCESS
3.1 Evaluation and Selection of Specifications / Features
3.2 Design Constraints
3.3 Analysis and Finalization of Features
3.4 Design Flow
3.5 Design Selection
3.6 Implementation Plan / Methodology
CHAPTER 4. RESULTS ANALYSIS AND VALIDATION
4.1 Implementation of Solution
CHAPTER 5. CONCLUSION AND FUTURE WORK
5.1 Conclusion
5.2 Future Work
REFERENCES
APPENDIX
USER MANUAL

LIST OF FIGURES

Figure 3.1 Student Management System flowchart (textual flow)

Figure 3.2 Main menu flow

Page | 9

Figure 4.1 Sample console interaction

LIST OF TABLES

Table 3.1 Feature evaluation

Table 3.2 Design alternatives comparison

Page | 10

Table 4.1 Test cases and expected outcomes

CHAPTER 1.
INTRODUCTION

1.1 Identification of Client / Need / Relevant Contemporary Issue

Educational institutions maintain student information for academic and administrative purposes. Common details include roll number, student name, course, and marks. When such information is managed only through paper registers or separate files, finding a particular record or correcting an error may take additional time. Duplicate entries and inconsistent updates may also occur.

The need addressed by this project is a small, easy-to-understand application that demonstrates organized student record management. The intended user is a student, teacher, or office assistant working with a limited number of sample records. This report does not claim that a formal institutional survey was conducted; the need is identified from the common limitations of manual record keeping and the educational objective of applying OOP concepts.

1.2 Identification of Problem

The main problem is the lack of a simple, centralized process for storing and managing student records in a small academic environment. The user needs to enter records, view the available records, find a student using a roll number, modify incorrect details, and remove a record when required. A solution should also reduce repeated code and keep each student's data grouped together.

1.3 Identification of Tasks

1. Identify the information required for each student record.
2. Design a Student class with data members and member functions.
3. Develop a menu-driven interface for user interaction.
4. Implement add, display, search, update, and delete operations.
5. Use file handling to preserve records between program runs.
6. Test the main functions with valid, missing, and repeated roll numbers.
7. Document the implementation, limitations, and steps for running the program.

1.4 Timeline

The following is a proposed project timeline for an individual academic project. The actual duration may vary according to course deadlines.

Week	Activity	Deliverable
Week 1	Requirement analysis and planning	Feature list
Week 2	Class design and menu structure	Program outline
Week 3	Coding and file handling	Working prototype
Week 4	Testing and corrections	Test results
Week 5	Report preparation	Final report

1.5 Organization of the Report

Chapter 1 introduces the need, problem, tasks, timeline, and report structure. Chapter 2 discusses the background and common approaches to student record management. Chapter 3 describes the required features, constraints, alternative designs, and selected implementation method. Chapter 4 explains the program implementation and validation using test cases. Chapter 5 summarizes the project and suggests future improvements. The final sections contain references, appendices, and the user manual.

CHAPTER 2.

LITERATURE REVIEW / BACKGROUND STUDY

2.1 Timeline of the Reported Problem

Record management has traditionally been performed with paper registers, forms, and filing systems. As computers became widely available, institutions began using spreadsheets and database applications to store and retrieve records. Desktop applications and web-based student information systems later combined record storage with features such as attendance, examination results, and reporting.

This project is a small educational example rather than a historical or statistical study. It focuses on the programming concepts required to implement basic record operations in C++.

2.2 Existing Solutions

Manual registers are simple and do not require a computer, but searching and updating records can be slow. Spreadsheets provide sorting and filtering but depend on careful editing and may not enforce application-specific rules. Database-based student information systems can support many users and larger datasets, but they require more setup and knowledge of database design. A console-based C++ application is smaller and more suitable for demonstrating classes, functions, conditions, loops, and file operations.

2.3 Feature-Based Review

Approach	Strength	Limitation	Suitability
Paper register	Low cost; easy to start	Slow search and editing	Very small records
Spreadsheet	Sort and filter features	Manual validation; accidental changes	Small to medium records
Database system	Structured queries and stronger integrity	Needs database setup and administration	Larger systems
C++ console application	OOP practice; custom menu and file storage	Basic interface; limited multi-user support	Learning project

2.4 Review Summary

The review indicates that the required solution should be easy to run, have a clear menu, group student data in a class, and provide core record operations. For this project, a console interface and local file storage provide a reasonable balance between simplicity and usefulness.

2.5 Problem Definition

The project will create a C++ program that stores student roll number, name, course, and marks. It will provide functions to add a record, display all records, search by roll number, update a record, and delete a record. The program will use a local text file for persistence. It will not include institutional login, network access, multi-user concurrency, or integration with a real university database.

2.6 Goals / Objectives

- Apply the principles of object-oriented programming in a practical example.
- Represent each student using a class and an object-oriented data structure.
- Provide a menu-driven and understandable interface.
- Implement the main CRUD operations on student records.
- Save and reload records using file handling.
- Validate common input conditions and test the program's expected behaviour.

CHAPTER 3.

DESIGN FLOW / PROCESS

3.1 Evaluation and Selection of Specifications / Features

The main specifications are simplicity, readable code, reliable record identification, and persistence. Roll number is used as the unique identifier in the program. Student name and course are stored as text, while marks are stored as a numeric value. The menu gives each operation a separate option.

Feature	Reason	Priority
Add student	Create a new student record	Essential
Display students	Review all saved records	Essential
Search by roll number	Find a specific record quickly	Essential
Update student	Correct existing information	Essential
Delete student	Remove an unwanted record	Essential
File storage	Keep records after exit	Essential
Graphical interface	Improve visual interaction	Optional future work

3.2 Design Constraints

- Platform: Standard C++ compiler supporting file streams and common standard-library features.
- Interface: Console-based; no external graphical framework is required.
- Storage: A local text file is used; the file must be accessible with read/write permissions.
- Data validation: Roll numbers should be unique, and marks should be within 0 to 100.
- Security: This prototype does not include encryption, user accounts, or access control; it should not store sensitive real-world data.
- Scalability: The design is intended for small learning datasets, not a high-volume institutional system.

3.3 Analysis and Finalization of Features

The final feature set includes add, display, search, update, delete, and file persistence. Features such as attendance, fees, course registration, authentication, and report generation are not included in the core version because they would increase the scope beyond the main OOP learning objectives.

3.4 Design Flow

START → Load records from file → Show menu → Read choice

↓

Add / Display / Search / Update / Delete / Exit

↓

Validate input → Perform operation → Save changes → Return to menu

↓

EXIT

Figure 3.1: Textual flow of the Student Management System.

3.5 Design Selection

Design alternative	Advantages	Decision
Option A: Simple menu with records only in memory	Easy to implement; no file handling	Not selected because records are lost after exit
Option B: Menu, Student class, and file storage	Demonstrates OOP and keeps records between runs	Selected for this project
Option C: GUI with database	More user-friendly and scalable	Not selected because it needs extra tools and setup

Option B is selected because it covers the subject's key programming concepts while remaining manageable for a student project. The design is modular enough to be extended later.

3.6 Implementation Plan / Methodology

8. Define the Student class and its data members.
9. Create functions to load records from the data file and save records to it.
10. Implement the menu and use a loop to keep the application running until Exit is selected.
11. Implement record operations and check the roll number before adding, updating, or deleting.
12. Compile the program, correct errors, and run test cases for normal and invalid inputs.
13. Prepare documentation and a user manual.

CHAPTER 4.

RESULTS ANALYSIS AND VALIDATION

4.1 Implementation of Solution

The program uses a Student class to group related information and a StudentManager class to manage the collection of records. The vector stores the records while the text file provides persistence. Member functions keep the program organized and reduce repetition. The menu uses a switch statement to call the requested operation.

Program source code

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <limits>
using namespace std;

class Student {
public:
    int rollNo;
    string name, course;
    float marks;

    Student() : rollNo(0), marks(0) {}
    Student(int r, string n, string c, float m)
        : rollNo(r), name(n), course(c), marks(m) {}
};

class StudentManager {
private:
    vector<Student> students;
    const string fileName = "students.txt";

    int findIndex(int roll) {
        for (int i = 0; i < (int)students.size(); i++)
            if (students[i].rollNo == roll) return i;
        return -1;
    }

    void loadFromFile() {
        ifstream fin(fileName);
        if (!fin) return;
        students.clear();
        string line;
        while (getline(fin, line)) {
            stringstream ss(line);
            string r, n, c, m;
            if (getline(ss, r, '|') && getline(ss, n, '|') &&
                getline(ss, c, '|') && getline(ss, m)) {
                try {
                    students.emplace_back(stoi(r), n, c, stof(m));
                } catch (...) {
                    // Ignore a malformed line in the data file.
                }
            }
        }
    }

    void saveToFile() {
```

```
        ofstream fout(fileName);
        for (const Student &s : students)
            fout << s.rollNo << '|' << s.name << '|'
                 << s.course << '|' << s.marks << '\n';
    }

    int readInt(const string &prompt) {
        int value;
        while (true) {
            cout << prompt;
            if (cin >> value) {
                cin.ignore(numeric_limits<streamsize>::max(), '\n');
                return value;
            }
            cout << "Invalid input. Enter a whole number.\n";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
        }
    }

    float readMarks() {
        float value;
        while (true) {
            cout << "Enter marks (0-100): ";
            if (cin >> value && value >= 0 && value <= 100) {
                cin.ignore(numeric_limits<streamsize>::max(), '\n');
                return value;
            }
            cout << "Invalid marks. Enter a value from 0 to 100.\n";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
        }
    }

public:
    StudentManager() { loadFromFile(); }

    void addStudent() {
        int roll = readInt("Enter roll number: ");
        if (findIndex(roll) != -1) {
            cout << "A student with this roll number already exists.\n";
            return;
        }
        string name, course;
        cout << "Enter student name: ";
        getline(cin, name);
        cout << "Enter course: ";
        getline(cin, course);
        float marks = readMarks();
        students.emplace_back(roll, name, course, marks);
        saveToFile();
        cout << "Student added successfully.\n";
    }

    void displayStudents() {
        if (students.empty()) {
            cout << "No student records found.\n";
            return;
        }
        cout << "\nROLL\tNAME\t\tCOURSE\t\tMARKS\n";
        cout << "-------------------------------------------------\n";
        for (const Student &s : students)
            cout << s.rollNo << "\t" << s.name << "\t\t"
                 << s.course << "\t\t" << s.marks << '\n';
    }

    void searchStudent() {
        int roll = readInt("Enter roll number to search: ");
        int i = findIndex(roll);
        if (i == -1) {
            cout << "Student not found.\n";
            return;
        }
        const Student &s = students[i];
```

```
        cout << "Roll number: " << s.rollNo << "\nName: " << s.name
             << "\nCourse: " << s.course << "\nMarks: " << s.marks << '\n';
    }

    void updateStudent() {
        int roll = readInt("Enter roll number to update: ");
        int i = findIndex(roll);
        if (i == -1) {
            cout << "Student not found.\n";
            return;
        }
        cout << "Enter new student name: ";
        getline(cin, students[i].name);
        cout << "Enter new course: ";
        getline(cin, students[i].course);
        students[i].marks = readMarks();
        saveToFile();
        cout << "Student updated successfully.\n";
    }

    void deleteStudent() {
        int roll = readInt("Enter roll number to delete: ");
        int i = findIndex(roll);
        if (i == -1) {
            cout << "Student not found.\n";
            return;
        }
        students.erase(students.begin() + i);
        saveToFile();
        cout << "Student deleted successfully.\n";
    }

    void run() {
        int choice;
        do {
            cout << "\n===== STUDENT MANAGEMENT SYSTEM =====\n"
                 << "1. Add Student\n2. Display Students\n"
                 << "3. Search Student\n4. Update Student\n"
                 << "5. Delete Student\n6. Exit\n";
            choice = readInt("Enter your choice: ");
            switch (choice) {
                case 1: addStudent(); break;
                case 2: displayStudents(); break;
                case 3: searchStudent(); break;
                case 4: updateStudent(); break;
                case 5: deleteStudent(); break;
                case 6: cout << "Program closed.\n"; break;
                default: cout << "Invalid choice. Select 1 to 6.\n";
            }
        } while (choice != 6);
    }
};

int main() {
    StudentManager system;
    system.run();
    return 0;
}
```

Program explanation

1. Student class: stores roll number, name, course, and marks for one student. Constructors initialize these values.

2. StudentManager class: manages the collection of students and implements the operations. Its helper functions locate a record, load the file, save the file, and validate numeric input.

3. File handling: students.txt stores one record per line. The vertical bar character separates the four fields. The file is read when the program starts and rewritten after changes.

4. Menu: the do-while loop repeatedly displays options. The switch statement directs the chosen option to the relevant member function.

5. Validation: duplicate roll numbers are rejected, marks are limited to 0–100, and invalid integer input is handled without terminating the program.

Sample console interaction

```
===== STUDENT MANAGEMENT SYSTEM =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice: 1
Enter roll number: 101
Enter student name: Aanya Sharma
Enter course: BCA Data Science
Enter marks (0-100): 88
Student added successfully.
```

The interaction above is an illustrative example of the expected console output. Actual output depends on the user's input and the records in students.txt.

Testing and validation

Test case	Input / action	Expected result	Status
Add valid record	Unique roll number; marks 0–100	Record saved	To be verified by running
Duplicate roll number	Use an existing roll number	Duplicate rejected	To be verified by running
Display records	Choose option 2	Saved records displayed	To be verified by running
Search existing record	Enter an existing roll number	Matching details displayed	To be verified by running
Search missing record	Enter unknown roll number	Not-found message displayed	To be verified by running
Invalid marks	Enter marks below 0 or above 100	Program asks again	To be verified by running
Update record	Enter existing roll number and new details	Changes saved	To be verified by running
Delete record	Enter existing roll number	Record removed and saved	To be verified by running
Exit	Choose option 6	Program closes	To be verified by running

Note: The test table lists planned validation cases. The report does not claim that these tests have been executed on a particular machine; the student should compile and run the program, then update the status column based on actual results.

CHAPTER 5.
CONCLUSION AND FUTURE WORK

5.1 Conclusion

The Student Management System in C++ demonstrates how object-oriented programming can be applied to a practical record-management task. The application groups student information in a class and uses member functions to add, display, search, update, and delete records. A menu-driven interface makes the operations easy to follow, while file handling allows records to be retained between program runs.

The project supports learning outcomes related to classes, objects, constructors, encapsulation, functions, vectors, conditional statements, loops, exception handling, and file streams. Its main limitation is that it is a basic single-user console prototype. It has not been integrated with an institutional database or tested as a production system.

5.2 Future Work

- Add login authentication and role-based access for administrator, teacher, and student users.
- Replace text-file storage with a relational database such as MySQL or SQLite.
- Create a graphical user interface for easier navigation.
- Add attendance, subject-wise marks, grade calculation, and report generation.
- Improve validation for empty names, separator characters in text fields, and malformed data files.
- Add automated unit tests and test the program across different compilers and operating systems.
- Support data backup, import/export, and secure handling of personal information.

REFERENCES

14. Bjarne Stroustrup, Programming: Principles and Practice Using C++, 2nd edition, Addison-Wesley, 2014.
15. cppreference.com, “C++ language and standard library reference.” Available at: https://en.cppreference.com/ (accessed October 2026).
16. ISO C++ Foundation, “Get Started with C++.” Available at: https://isocpp.org/get-started (accessed October 2026).
17. Chandigarh University, Project Report Format supplied for the Object Oriented Programming Language project.

APPENDIX

Appendix A: Design Checklist

- Project title and student identification included.
- Problem statement and objectives described.
- Core system features identified.
- Class-based design and file handling explained.
- Source code included in the report.
- Sample interaction and planned test cases included.
- Limitations and future improvements documented.

Appendix B: Plagiarism Report

Attach the plagiarism/similarity report required by the course or department here, if applicable. No similarity score is claimed in this document.

Appendix C: Design notes

Data file format: roll number|student name|course|marks. Each student is stored on a separate line. Do not manually include the separator character '|' inside a name or course because it is used to split the fields.

USER MANUAL

System requirements

- A computer with a C++ compiler such as GCC, Clang, or a compatible IDE.
- Support for standard C++ file streams and the vector and string libraries.
- Permission to create and update students.txt in the program's working directory.

Steps to compile and run

18. Open a C++ IDE or a text editor and create a file named student_management.cpp.
19. Copy the source code from Chapter 4 into the file.
20. Compile with a C++11-or-later compiler. For GCC, use: g++ student_management.cpp -o student_management
21. Run the executable. On Windows, run student_management.exe; on Linux or macOS, run ./student_management.
22. Choose an option from 1 to 6 when the menu appears.
23. Use Add Student to enter a unique roll number, name, course, and marks.
24. Use Display Students to view all records; Search Student to find a roll number.
25. Use Update Student to replace a student's name, course, and marks; use Delete Student to remove a record.
26. Choose Exit to close the program. Records are stored in students.txt in the working directory.

Troubleshooting

If compilation fails, check that all source code has been copied and that the compiler supports C++11 or later. If records do not appear after restarting, check that students.txt is in the program's current working directory and that the application has permission to read it. If a roll number already exists, use a different number or update the existing record.

Important note

This is an academic demonstration program. Use sample data while learning. Before using a similar system for real student information, add authentication, appropriate access controls, backups, privacy safeguards, and stronger data validation.